

VELOCITY GLOBAL SOLUTIONS

Technical Hiring Assessment

Task Role: Front End Developer | **Deadline:** 25/03/2026, 11:59 PM IST | **AI Tools:** Strictly Prohibited

Task: Build a Multi-View Project Tracker UI with Custom Drag-and-Drop, Virtual Scrolling & Live Collaboration Indicators

Overview Build a fully functional frontend application for a project management tool. The interface must support three different views of the same data, a custom-built drag-and-drop system (no libraries), virtual scrolling for large lists, and real-time collaboration indicators. This task tests genuine frontend engineering — layout logic, performance thinking, state architecture, and UI precision — none of which AI tools reliably produce together in a working state.

What You Must Build**1. Three Views of the Same Task Data**

The application must display one shared dataset in three switchable views. Switching between views must be instant — no re-fetch, same state:

Kanban Board View

- Four columns: To Do, In Progress, In Review, Done
- Tasks displayed as cards within each column
- Column task count shown in the header
- Cards must show: title, assignee avatar (initials-based, no images), priority badge (colour-coded), due date with overdue highlighting in red
- Columns must scroll independently when cards overflow

List View

- All tasks in a flat, sortable table
- Sortable by: title (A–Z), priority (Critical → Low), due date (earliest first)
- Each column header must be clickable to toggle sort direction — active sort column must be visually indicated
- Multi-sort not required — single active sort only
- Rows must support inline status change via a dropdown directly in the row — no modal required

Timeline / Gantt View

- Tasks plotted on a horizontal time axis spanning the current month
- Each task rendered as a horizontal bar from its start date to due date
- Bars must be colour-coded by priority

- Today's date must be marked with a visible vertical line
- Tasks with no start date must still appear as a single-day marker on their due date
- The timeline must be horizontally scrollable if it overflows

2. Custom Drag-and-Drop (No Libraries)

This is the hardest part of the task. You must implement drag-and-drop from scratch using native browser drag events or pointer events — no react-beautiful-dnd, no dnd-kit, no SortableJS, no external drag library of any kind.

- Drag cards between Kanban columns to change their status
- When a card is dragged, the original position must show a placeholder of the same height
- The dragged card must follow the cursor with a slight opacity reduction and a drop shadow
- Valid drop zones must be visually indicated (subtle background colour change) when a card is dragged over them
- If a card is dropped outside a valid column, it must snap back to its original position with a smooth transition
- The drag must work correctly on both mouse and touch devices

3. Virtual Scrolling in List View

The list view must handle large datasets without performance degradation. Implement virtual scrolling from scratch — no react-window, no react-virtualized, no virtual scrolling library:

- Only render the rows currently visible in the viewport plus a buffer of 5 rows above and below
- Total row count and scroll position must remain correct even when only a subset of rows are in the DOM
- Scrolling must feel smooth — no flickering, no blank gaps, no jump on fast scroll
- Test your implementation with a dataset of at least 500 tasks (include a data generator in your seed file)

4. Live Collaboration Indicators

Simulate real-time presence without a backend — use a mock WebSocket or a simple interval-based simulation:

- Show 2–4 simulated "other users" who are randomly viewing or editing tasks
- Each active user has a coloured avatar indicator shown on the task card they are currently viewing
- When a simulated user moves to a different task, the indicator must animate off one card and onto another
- At the top of the screen, show an avatars bar: "3 people are viewing this board" — update it as simulated users move
- If two simulated users are on the same card, stack their avatars with a +1 style overflow indicator

5. Filters & URL State

- Filter bar above all views with: status (multi-select), priority (multi-select), assignee (multi-select), due date range (from / to date inputs)

- Filters must apply instantly — no submit button
- Active filters must be reflected in the URL as query parameters so the filtered view is shareable and bookmarkable
- Navigating back must restore the exact filter state from the URL
- A clear all filters button must appear only when at least one filter is active

6. Empty States & Edge Cases

- Each Kanban column with no tasks must show a styled empty state — not a blank column
- List view with no results after filtering must show a meaningful empty state with a "Clear filters" CTA
- Tasks with a due date of today must be labelled "Due Today" instead of the date
- Tasks overdue by more than 7 days must show the number of days overdue, not the date

Technical Requirements

- Framework: React with TypeScript — no plain JavaScript accepted
- Styling: Tailwind CSS or CSS Modules — no component libraries (no MUI, no Ant Design, no Chakra) — build your own components
- State Management: Zustand or React Context + useReducer — justify your choice in the README
- No drag-and-drop libraries — this will be checked
- No virtual scrolling libraries — this will be checked
- No UI component libraries — buttons, modals, dropdowns, badges must all be custom built
- Responsive: Must work correctly on desktop (1280px+) and tablet (768px) — mobile not required
- Performance: Lighthouse performance score must be 85 or above on desktop — include a screenshot of your Lighthouse report in the README

Seed Data Required Include a data generator that produces at least 500 tasks with randomised titles, assignees (from a pool of 6 users), priorities, statuses, start dates, and due dates — including some overdue tasks and some with missing start dates to test edge cases.

Submission Requirements

- Public GitHub/GitLab repository with a live deployed link (Vercel or Netlify — free tier is fine)
- README must include: setup instructions, state management decision justification, explanation of how you implemented virtual scrolling, explanation of your drag-and-drop approach, Lighthouse screenshot
- In the Explanation field (150–250 words): the hardest UI problem you solved, how you handled the drag placeholder without layout shift, and one thing you'd refactor with more time

Evaluation

Criteria	Weight
Custom drag-and-drop — correct behaviour, touch support, snap back	25%
Virtual scrolling — smooth, correct row count, no blank gaps at 500+ rows	25%
Three views — correct data, instant switching, no re-fetch	20%
URL-synced filters — shareable, restores on back navigation	15%
Code quality — TypeScript usage, component structure, no library shortcuts	15%

Auto-Disqualification: Any drag-and-drop library detected, any virtual scrolling library detected, any UI component library detected, no live deployed link, no TypeScript, Lighthouse score below 85, plain JavaScript instead of TypeScript.

Completed the task? Submit your work here: (<https://bit.ly/4bGXmZV>)